

Sistema RAG con Google Drive y Supabase

Automatización de ingesta documental y consulta inteligente mediante agente conversacional (n8n)

Resumen del proyecto

Este proyecto implementa un sistema de Retrieval Augmented Generation (RAG) compuesto por dos flujos complementarios en n8n: uno de ingesta automática de documentos y otro de consulta conversacional sobre esa información. El objetivo es que cualquier archivo agregado o modificado en una carpeta de Google Drive se procese, transforme en vectores y quede disponible para ser consultado en lenguaje natural por un agente de IA, sin intervención manual.

Flujo 1: Ingesta y actualización automática de documentos

Este flujo se dispara automáticamente ante dos eventos de Google Drive: creación de un archivo nuevo o actualización de uno existente. A partir de ahí, el proceso:

- Identifica el archivo modificado y descarga su contenido desde Drive.
- Elimina la fila correspondiente en la base de datos si el archivo ya existía, evitando información duplicada o desactualizada.
- Clasifica el archivo según su tipo (PDF, DOC, CSV, XLSX) mediante un nodo Switch, y lo deriva al extractor correspondiente.
- Extrae el texto o los datos según el formato: lectura de PDF, texto plano, CSV o Excel.
- Divide el contenido en fragmentos (chunks) mediante un Text Splitter, aplicando un tamaño de chunk (chunk size) y un solapamiento entre fragmentos (overlap) para preservar contexto entre bloques consecutivos.
- Genera los embeddings de cada fragmento con OpenAI Embeddings.
- Guarda el vector resultante en la tabla de Supabase (pgvector), quedando disponible para su consulta.

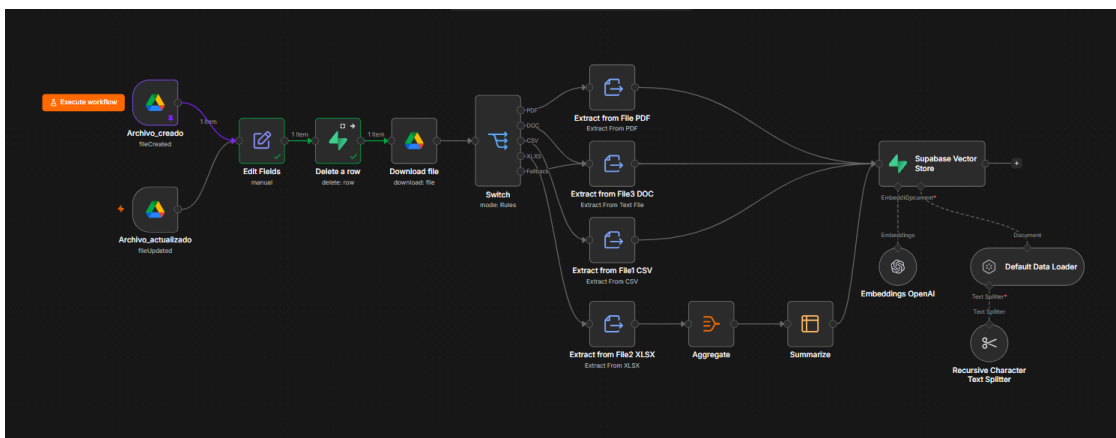


Figura 1. Flujo de ingesta automática desde Google Drive hacia Supabase Vector Store

Conceptos técnicos clave

Chunk size: tamaño de cada fragmento de texto en el que se divide un documento antes de generar su embedding. Un tamaño adecuado balancea precisión semántica (fragmentos chicos, más específicos) contra contexto suficiente (fragmentos grandes, con más información conjunta).

Overlap: cantidad de texto que se repite entre un chunk y el siguiente. Evita que una idea quede cortada exactamente en el límite entre dos fragmentos, mejorando la calidad de las respuestas al recuperar información.

Deduplicación: antes de volver a insertar un archivo actualizado, el flujo elimina su versión anterior en la base, garantizando que el agente siempre consulte la información más reciente y no arrastre datos obsoletos o repetidos.

Flujo 2: Agente conversacional de consulta (RAG)

Este segundo flujo expone un agente de IA que responde preguntas en lenguaje natural utilizando la información previamente indexada. Su arquitectura combina:

- Un disparador de tipo chat (When chat message received) que recibe la consulta del usuario.
- Un AI Agent (AI Agent RAG) que orquesta el modelo de lenguaje (Google Gemini Chat Model), una memoria conversacional persistente en Postgres (Postgres Chat Memory) y una herramienta de búsqueda sobre la base de datos.
- La herramienta "Basededatos" consulta el Supabase Vector Store, que a su vez utiliza Embeddings OpenAI para transformar la pregunta del usuario en un vector y recuperar los fragmentos más relevantes por similitud semántica.
- Con esos fragmentos como contexto, el modelo Gemini redacta la respuesta final, manteniendo memoria de los intercambios anteriores dentro de la conversación.

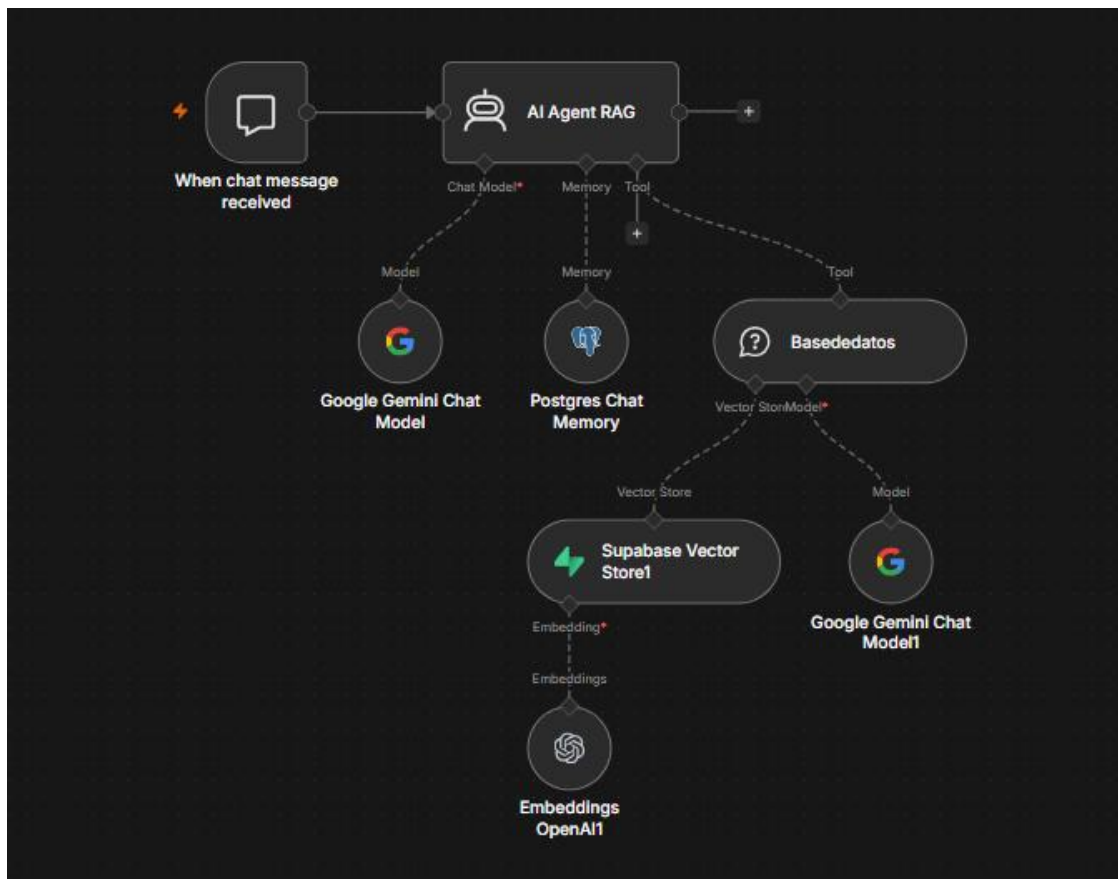


Figura 2. Agente conversacional RAG con memoria persistente y búsqueda vectorial

Configuración de la base de datos en Supabase

Antes de que los flujos de n8n pudieran escribir y leer vectores, fue necesario preparar la base en Supabase desde el SQL Editor:

- Se habilitó la extensión pgvector, que agrega el tipo de dato "vector" a Postgres y permite almacenar y comparar embeddings.

- Se creó la tabla documents con las columnas id, content (el texto del chunk), metadata (información adicional en formato jsonb, como el archivo de origen y el rango de líneas) y embedding (el vector de 1536 dimensiones, la dimensión estándar de los embeddings de OpenAI).
- Se definió una función match_documents encargada de recibir un vector de consulta y devolver los documentos más similares, que es la que utiliza el nodo Supabase Vector Store del agente para responder preguntas.

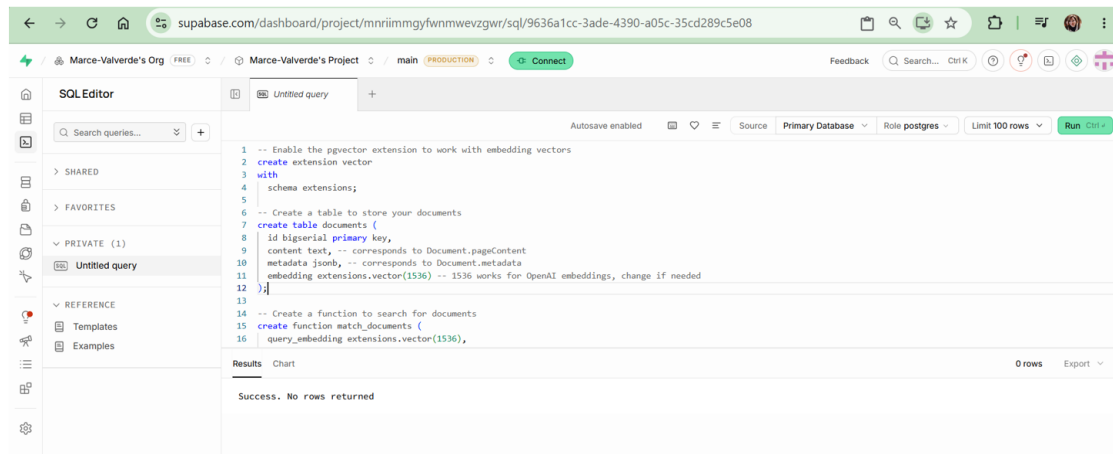


Figura 3. Script SQL: extensión pgvector y creación de la tabla documents

Al principio la tabla queda vacía, a la espera de que el flujo de ingesta la alimente:

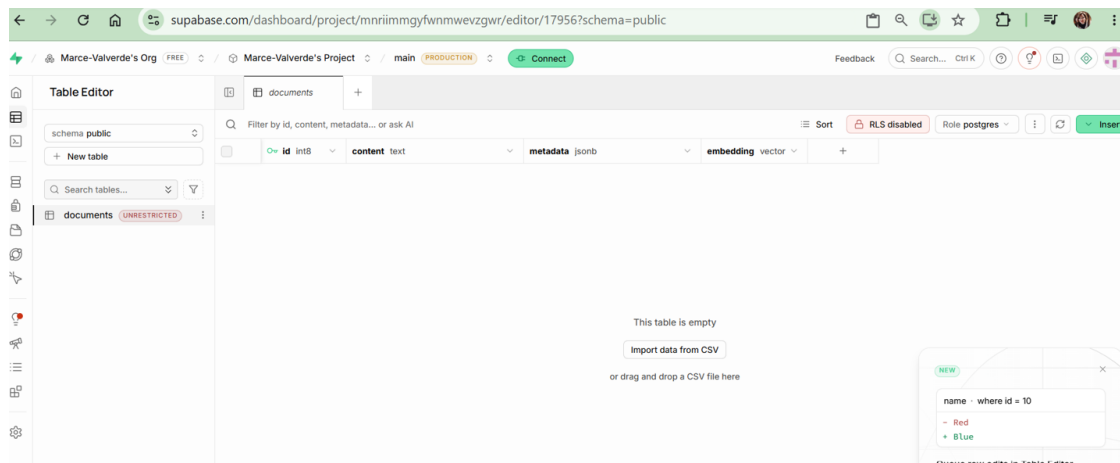


Figura 4. Tabla documents recién creada, sin registros

Una vez que el flujo de ingesta procesa los archivos de Drive, cada chunk de texto queda registrado como una fila independiente, junto con su metadata (posición dentro del documento original) y su embedding correspondiente:

id	content	metadata	embedding
1	1.4	{ "loc": { "lines": { "to": "1", "from": "1" }, "line": "1" }	[0.03262329, -0.02120]
2	es	{ "loc": { "lines": { "to": "1", "from": "1" }, "line": "2" }	[-0.0025863647, 0.025]
3	Documento sin título	{ "loc": { "lines": { "to": "1", "from": "1" }, "line": "3" }	[-0.0020160675, 0.030]
4	Skia/PDF m151 Google Docs Renderer	{ "loc": { "lines": { "to": "1", "from": "1" }, "line": "4" }	[-0.019500732, 0.0445]
5	Guía Estratégica para Crear una Agenc	{ "loc": { "lines": { "to": "5", "from": "1" }, "line": "4" }	[0.008232117, 0.035125]
6	fundamental priorizar los fundamentos	{ "loc": { "lines": { "to": "10", "from": "6" }, "line": "4" }	[-0.0049438477, 0.020]
7	2. El Valor de "Lo Aburrido" y el Pensar	{ "loc": { "lines": { "to": "14", "from": "10" }, "line": "4" }	[0.0046958923, 0.0193]
8	Para gestionar cualquier proceso, se di	{ "loc": { "lines": { "to": "19", "from": "14" }, "line": "4" }	[-0.003768921, 0.0584]
9	alcanzar el objetivo. 3. Paso 1: Selecció	{ "loc": { "lines": { "to": "25", "from": "19" }, "line": "4" }	[0.027328491, -0.0145]
10	estéticas de alto valor, bienes raíces).	{ "loc": { "lines": { "to": "31", "from": "25" }, "line": "4" }	[0.0027580261, 0.0237]
11	ventas pequeñas para un negocio de b	{ "loc": { "lines": { "to": "33", "from": "31" }, "line": "4" }	[-0.017242432, 0.0222]
12	especialización aumenta el valor percib	{ "loc": { "lines": { "to": "39", "from": "35" }, "line": "4" }	[0.0079574585, 0.0098]
13	que cobra por resultados. Si tu solució	{ "loc": { "lines": { "to": "46", "from": "40" }, "line": "4" }	[-0.009521484, 0.0055]
14	optimización y psicología del cliente.	{ "loc": { "lines": { "to": "52", "from": "46" }, "line": "4" }	[-0.0115356445, 0.0183]
15	zona de confort. • Métodos de prospec	{ "loc": { "lines": { "to": "60", "from": "52" }, "line": "4" }	[0.018081665, 0.03079]

Figura 5. Tabla documents poblada con los fragmentos (chunks) extraídos y sus embeddings

Memoria del agente: historial de conversación

Además de la tabla documents, el nodo Postgres Chat Memory del agente utiliza su propia tabla, n8n_chat_histories, para persistir el historial de cada conversación. Cada fila guarda un mensaje (humano, IA o llamada a herramienta) asociado a un session_id, que identifica a qué conversación pertenece. Esto permite que el agente recuerde el contexto de intercambios anteriores dentro de una misma sesión, en lugar de responder cada pregunta de forma aislada.

La memoria Postgres del agente AI RAG me permite guardar un historial de la conversación.

id	session_id	message
1	c22e7658fd4cfca068e67137363_	{ "type": "human", "content": "que es tele" }
2	c22e7658fd4cfca068e67137363_	{ "type": "ai", "content": "Calling Basedec" }
3	c22e7658fd4cfca068e67137363_	{ "name": "Basedatos", "type": "tool", "c" }
4	c22e7658fd4cfca068e67137363_	{ "type": "ai", "content": "Lo siento, actua" }

Figura 6. Tabla n8n_chat_histories con el historial de mensajes de una conversación

Stack tecnológico

Componente	Herramienta / Servicio
Orquestación	n8n
Origen de datos	Google Drive (triggers: fileCreated, fileUpdated)
Base de datos vectorial	Supabase (pgvector), vía Session Pooler para conexión IPv4
Embeddings	OpenAI Embeddings
Modelo conversacional	Google Gemini Chat Model

Memoria de conversación	Postgres Chat Memory
Formatos soportados	PDF, DOC/TXT, CSV, XLSX

Resultado

El sistema mantiene una base de conocimiento siempre actualizada a partir de una carpeta de Drive, sin pasos manuales de carga, evitando duplicados y permitiendo que un agente conversacional responda preguntas sobre esos documentos con contexto y memoria de la conversación. Esta arquitectura es la base para automatizar atención al cliente, soporte interno o asistentes de conocimiento para pequeñas empresas — uno de los pilares de servicios que ofrece MVImpulsa.